Übungsaufgaben: Exakte Algorithmen und ETH

Algorithmen und Komplexität

Aufgabe 1: Ada und die größte Happy Group

Ada möchte möglichst viele Gäste zu ihrer Party einladen. Allerdings gibt es einige Paare von Gästen, die sich nicht ausstehen können. Von jedem solchen Paar darf höchstens eine Person eingeladen werden.

Die Situation wird durch einen ungerichteten Graphen $G = (V, E)$ mit $n = |V|$ beschrieben: Die Knoten entsprechen den Gästen, und eine Kante $\{u, v\} \in E$ bedeutet, dass sich u und v nicht vertragen.

Eine Teilmenge $W \subseteq V$, in der sich alle Gäste miteinander vertragen, nennen wir eine *Happy Group* (in der Graphentheorie auch als *unabhängige Menge* bekannt).

(a) Beschreiben Sie einen Brute-Force-Algorithmus, der eine Happy Group maximaler Größe findet, und bestimmen Sie seine Laufzeit.

Ada teilt die Gäste in zwei Gruppen $V = A \dot\cup B$. Innerhalb der Gruppe B gibt es keine Streitigkeiten, d. h., B ist eine Happy Group.

Angenommen, Ada hat sich bereits für eine Happy Group $I_A \subseteq A$ entschieden, deren Mitglieder sie auf jeden Fall einladen möchte.

Welche Gäste aus B kann sie zusätzlich einladen?

(b) Entwickeln Sie einen exakten Algorithmus, der für eine gegebene Zerlegung $V = A \dot\cup B$, bei der B eine Happy Group ist, eine Happy Group maximaler Größe in G in Zeit

$$O(n^2 2^k)$$

findet, wobei $k = |A|$ gilt.

Begründen Sie die Korrektheit und die Laufzeit Ihres Algorithmus.

(c) Angenommen, die Voraussetzungen aus (c) gelten und zusätzlich ist $|A| = |B| = n/2$.

Welche Laufzeit ergibt sich für Ihren Algorithmus? Vergleichen Sie diese mit der Laufzeit des Brute-Force-Algorithmus aus (a).

Aufgabe 2: Berts verrückte Bibliothek

Bert möchte n Bücher in die richtige Reihenfolge $1, 2, \ldots, n$ bringen.

Die Bücher stehen zunächst in einer beliebigen Reihenfolge, beschrieben durch eine Permutation $\pi = (\pi_1, \ldots, \pi_n)$.

Leider gelten in der Bibliothek strenge bürokratische Vorschriften: Laut Bücher-Umsortierungsverordnung (BUmsV) darf Bert in einem Schritt ausschließlich die Reihenfolge eines zusammenhängenden Abschnitts der Bücher umkehren. Alle anderen Umordnungen bedürfen einer gesonderten Genehmigung.

Beispielsweise kann Bert die Reihenfolge

$$(4, 3, 2, 1, 6, 5)$$

durch zwei aufeinanderfolgende Umkehrungen sortieren:

$$\begin{aligned}(4, 3, 2, 1, 6, 5) &\longrightarrow (1, 2, 3, 4, 6, 5)\\ &\longrightarrow (1, 2, 3, 4, 5, 6).\end{aligned}$$

Dabei wird im ersten Schritt der gesamte Abschnitt $(4, 3, 2, 1)$ umgekehrt und im zweiten Schritt der Abschnitt $(6, 5)$. Eine Umkehroperation vertauscht also nicht nur die beiden Bücher an den Enden des Abschnitts, sondern kehrt die Reihenfolge aller Bücher innerhalb dieses Abschnitts um.

Bert möchte wissen, ob er die Bücher mit *höchstens k Umkehrungen* sortieren kann.

(a) Wie viele verschiedene Umkehroperationen kann Bert bei n Büchern ausführen, wenn der umgekehrte Abschnitt aus mindestens zwei Büchern bestehen muss?

Entwickeln Sie einen Algorithmus, der durch systematisches Ausprobieren entscheidet, ob Bert die Bücher mit höchstens k Umkehrungen sortieren kann.

Hinweis: Überlegen Sie, welche Bücherreihenfolgen nach $0, 1, 2, \ldots, k$ Umkehrungen erreichbar sind.

(b) Bert hat eine Idee: Er könnte nicht nur von der Anfangsreihenfolge aus suchen, sondern auch von der gewünschten Endreihenfolge aus.

Entwickeln Sie daraus einen Algorithmus, der das Problem ebenfalls korrekt entscheidet, aber im Allgemeinen deutlich weniger Möglichkeiten untersucht.

Hinweis: Jede Umkehroperation lässt sich rückgängig machen. Könnte es daher genügen, von beiden Seiten aus nach einer gemeinsamen Zwischenreihenfolge zu suchen?

(c) Vergleichen Sie die Laufzeiten beider Algorithmen in Abhängigkeit von n und k.

Aufgabe 3: Der billigste Weg durch alle Städte

Clara möchte n Städte besuchen, wobei jede Stadt genau einmal besucht werden soll.

Gegeben ist ein gerichteter Graph $G = (V, E)$ mit $n = |V|$ Knoten, $m = |E|$ Kanten und Kantengewichten $w : E \to \mathbb{N}$.

Gesucht ist ein *gerichteter hamiltonscher Pfad* $P = (v_1, v_2, \ldots, v_n)$ mit möglichst geringen Gesamtkosten $w(P) = \sum_{i=1}^{n-1} w(v_i, v_{i+1})$. Falls kein solcher Pfad existiert, soll dies erkannt werden.

(a) **Brute Force.**

Beschreiben Sie einen einfachen Algorithmus, der das Problem exakt löst. Bestimmen Sie seine Laufzeit.

(b) **Ein überraschend einfacher Spezialfall.**

Angenommen, das Streckennetz G ist ein *gerichteter azyklischer Graph (DAG)*, d. h., G enthält keine gerichteten Kreise.

(i) **Beweisen oder widerlegen Sie:** Es gibt DAGs, die zwei verschiedene gerichtete hamiltonsche Pfade besitzen.

(ii) Clara möchte weiterhin jede Stadt genau einmal besuchen und dabei möglichst geringe Reisekosten haben.

Zeigen Sie, dass man für einen DAG einen gerichteten hamiltonschen Pfad minimalen Gewichts (sofern einer existiert) in Zeit $O(n + m)$ finden kann, wobei $n = |V|$ und $m = |E|$ gilt.

Sie dürfen annehmen, dass G durch *Adjazenzlisten* dargestellt wird, d. h., für jeden Knoten $v \in V$ ist eine Liste $\text{Adj}(v) = \{u \in V \mid (v, u) \in E\}$ seiner direkten Nachfolger gespeichert.

Hinweis: Ein modifizierter Algorithmus zur Berechnung einer topologischen Sortierung könnte hilfreich sein.

(c) **Der allgemeine Fall.**

Für beliebige gerichtete Graphen möchte Clara einen Algorithmus entwickeln, der deutlich schneller als Brute Force ist.

Entwickeln Sie einen exakten Algorithmus mithilfe *dynamischer Programmierung*, der einen gerichteten hamiltonschen Pfad minimalen Gewichts findet oder feststellt, dass keiner existiert.

Beschreiben Sie dazu geeignete Teilprobleme, geben Sie eine Rekursionsgleichung einschließlich der Anfangswerte an und begründen Sie die Korrektheit.

Zeigen Sie, dass eine Laufzeit von $O(n^2 2^n)$ erreicht werden kann.

Aufgabe 4: Wie schnell ist schnell genug?

Die *Exponential Time Hypothesis (ETH)* besagt, dass 3-SAT nicht in Zeit $2^{o(n)}$ gelöst werden kann, wobei n die Anzahl der Variablen bezeichnet.

Wir betrachten 3-SAT-Instanzen mit n Variablen und $O(n)$ Klauseln.

Sei X ein NP-vollständiges Entscheidungsproblem. Die Eingabegröße einer Instanz von X bezeichnen wir mit N.

(a) **Exponentiell oder doch nicht?**

Angenommen, jemand entwickelt einen Algorithmus für X mit einer der folgenden Laufzeiten:

$$T_1(N) = N^{10}, \qquad T_2(N) = 2^{\sqrt{N}}, \qquad T_3(N) = 2^{N/1000},$$
$$T_4(N) = N^{10} \cdot 2^{N/\log N}, \quad T_5(N) = 2^{\sqrt{N}\log N}, \quad T_6(N) = 2^{N^2/\log N}.$$

Welche dieser Laufzeiten sind *subexponentiell*, d. h. von der Form $2^{o(N)}$?

Begründen Sie Ihre Antworten.

(b) **Eine kleine Reduktion.**

Angenommen, es gibt eine polynomielle Reduktion von 3-SAT auf X, die aus einer 3-SAT-Instanz mit n Variablen eine Instanz von X der Größe $N = O(n)$ erzeugt.

Für welche der hypothetischen Algorithmen $T_1, \ldots, T_6$ aus (a) folgt aus diesen Angaben ein Widerspruch zur ETH?

Begründen Sie Ihre Antworten.

Hinweis: Welche Laufzeit erhält man für 3-SAT, wenn man zuerst die Reduktion und anschließend den Algorithmus für X ausführt?

(c) **Eine größere Reduktion.**

Nun sei lediglich bekannt, dass die Reduktion Instanzen der Größe $N = O(n^2)$ erzeugt.

Untersuchen Sie erneut die Algorithmen mit Laufzeiten

$$T_2(N) = 2^{\sqrt{N}}, \quad T_4(N) = N^{10} \cdot 2^{N/\log N}, \quad T_7(N) = 2^{N^{1/3}}$$

Folgt aus der angegebenen Reduktionsgröße und der jeweiligen Laufzeit ein Widerspruch zur ETH? Begründen Sie Ihre Antworten.